

HOSPITAL PATIENT TRIAGE & BED ALLOCATOR

Operating Systems Lab — Semester Project Report

Group Members

Ahmad Ali 24F-0689
Muhammad Ahmad 24F-0513

Course: Operating Systems Lab

Submission Date: May 08, 2026

Table of Contents

1. Introduction	
1.1 Problem Statement	
1.2 Objectives	
1.3 System Overview	
2. System Design	
2.1 Architecture Overview	
2.2 Component Descriptions	
2.2.1 hospital.h — Shared Definitions	
2.2.2 Ward Configuration	
2.3 Data Flow	
3. Phase 1 — Shell Scripting	
3.1 Triage.sh	
3.1.1 Key Functions	
3.1.2 FIFO Write Logic	
3.1.3 Sample Run — Critical Patient	
3.1.4 Sample Run — Low Priority Patient	
3.2 start_hospital.sh	
3.3 stop_hospital.sh	
4. Phase 2 — Process Management & IPC	
4.1 Shared Memory Design	
4.1.1 Shared Memory Lifecycle	
4.2 FIFO Communication	
4.3 Priority-Based Scheduling	
4.3.1 Priority Aging	
4.4 Process Management (fork + execv)	
4.5 Scheduling Gantt Chart	
4.5.1 Average Waiting and Turnaround Times	
5. Phase 3 — Threading, Synchronisation & Semaphores	
5.1 Thread Design	
5.2 Mutex Usage	
5.3 Condition Variables	
5.4 Semaphore Design	
5.4.1 Named Semaphores (ICU and Isolation Capacity)	
5.4.2 Unnamed Semaphores (Bounded Producer-Consumer Queue)	
5.5 Blocked and Released Admission Scenario	
6. Testing	
6.1 Phase 1 Test Cases	

6.2 Phase 2 Test Cases.....

6.3 Phase 3 Test Cases.....

6.4 Valgrind Output.....

7. Challenges & Lessons Learned.....

7.1 Challenges.....

7.1.1 FIFO Blocking Behaviour.....

7.1.2 Race Condition on Discharge Routing

7.1.3 Shutdown Sequencing.....

7.1.4 Coalescing Index Management

7.1.5 Semaphore Normalisation of Bed Type

7.2 Lessons Learned

7.3 Future Improvements

Appendix A — Project File Structure

Appendix B — Key Constants (hospital.h)

1. Introduction

1.1 Problem Statement

Modern hospitals face complex challenges in patient management: coordinating admissions, tracking bed availability, prioritising critical cases, and ensuring safe transitions between departments. Manual or uncoordinated processes lead to delays, resource wastage, and potentially life-threatening oversights.

This project addresses these challenges by implementing a simulated Hospital Admissions Management System grounded in core operating systems concepts. The system replicates real-world hospital workflows using POSIX IPC, multi-threading, semaphore-controlled resource access, and dynamic memory allocation strategies.

1.2 Objectives

- Demonstrate practical application of IPC mechanisms: shared memory, FIFOs (named pipes), and semaphores.
- Implement priority-based patient scheduling with priority aging to prevent starvation.
- Use threads, mutexes, and condition variables for concurrent access to shared hospital resources.
- Apply best-fit memory allocation with coalescing for bed partition management.
- Simulate the full patient lifecycle: triage → admission → treatment → discharge.

1.3 System Overview

The system is composed of four cooperating components:

- `Triage.sh` — collects patient details, derives priority and bed type from severity, and sends formatted records over a named FIFO.
- `admissions` — the central hospital manager; reads triage records, manages the priority queue, allocates beds, forks patient simulators, and handles discharge.
- `patient_simulator` — simulates treatment by sleeping for a bed-type-dependent duration, then writes its patient ID to the discharge FIFO.
- `start_hospital.sh` / `stop_hospital.sh` — lifecycle scripts that initialise or tear down IPC resources and the admissions process.

The system is implemented across three phases, each introducing new OS concepts:

1. Phase 1 — Shell scripting (`Triage.sh`, lifecycle scripts)
2. Phase 2 — Process management, FIFOs, shared memory, and the priority queue
3. Phase 3 — Multi-threading, mutexes, condition variables, and semaphores

2. System Design

2.1 Architecture Overview

The system follows a producer-consumer multi-process architecture. The Triage script acts as the producer, writing patient records to the triage FIFO. The Admissions Manager acts as the consumer and orchestrator, reading records and managing the entire hospital workflow.

Key components and their roles:

Component	Type	Role
Triage.sh	Shell Script	Patient intake, priority computation, FIFO write
admissions	C Process	Queue management, bed allocation, child spawning, discharge
patient_simulator	C Process (child)	Treatment simulation, discharge FIFO write
WardStatus	Shared Memory	Global hospital state: partitions, queue, counters
triage_fifo	Named FIFO	Patient records from Triage to Admissions
discharge_fifo	Named FIFO	Discharge signals from Patient to Admissions
sem_icu_limit	Named Semaphore	Limit concurrent ICU admissions
sem_iso_limit	Named Semaphore	Limit concurrent Isolation admissions

2.2 Component Descriptions

2.2.1 hospital.h — Shared Definitions

The header file hospital.h defines all constants, data structures, and IPC parameters shared between components. Key structures include:

- **PatientRecord** — stores patient ID, name, age, severity, priority, care units, bed type, and arrival time.
- **BedPartition** — represents a segment of the ward's care-unit address space with a type, occupancy status, and patient assignment.
- **WardStatus** — the top-level shared memory structure holding the partition array, the priority queue, and summary counters.

2.2.2 Ward Configuration

Ward Zone	Beds	Care Units/Bed	Total Units	Semaphore Capacity
ICU	4	3	12	4
Isolation	4	2	8	4
General	12	1	12	—
Total	20	—	32	—

2.3 Data Flow

Patient data flows through the system in a unidirectional pipeline:

4. User runs Triage.sh with name, age, and severity.
5. Triage.sh computes priority and bed type; writes a pipe-delimited record to /tmp/triage_fifo.
6. The receptionist thread in admissions reads the record, parses it into a PatientRecord, and inserts it into the priority queue.
7. The scheduler thread dequeues the highest-priority patient, acquires the appropriate semaphore, allocates a bed partition (best-fit), and forks a patient_simulator process.
8. patient_simulator sleeps for the treatment duration, then writes its patient ID to /tmp/discharge_fifo.
9. The discharge listener thread reads the patient ID, routes it to the appropriate nurse thread.
10. The nurse thread calls free_bed(), which marks the partition as free, coalesces adjacent free partitions, and signals the bed_freed_cond condition variable.

3. Phase 1 — Shell Scripting

3.1 Triage.sh

Triage.sh is the patient intake script. It accepts three positional arguments — patient name, age, and severity (1–10) — and produces a formatted triage record.

3.1.1 Key Functions

Function	Purpose
validate_fun()	Validates name is non-empty; age is 1–150; severity is 1–10.
comp_priority()	Maps severity to priority: 9-10 → P1, 7-8 → P2, 5-6 → P3, 3-4 → P4, 1-2 → P5.
Bed_type()	Maps priority to bed type: P1-P2 → ICU, P3 → ISOLATION, P4-P5 → GENERAL.
get_care_unit()	Maps bed type to care units: ICU → 3, ISOLATION → 2, GENERAL → 1.

3.1.2 FIFO Write Logic

The script checks whether /tmp/triage_fifo exists as a named pipe using the -p test. If the admissions manager is running, the record is written directly to the FIFO; otherwise it is printed to stdout for debugging.

3.1.3 Sample Run — Critical Patient

```
./scripts/Triage.sh "Ali Khan" 25 9
```

Expected output:

```
=====
TRIAGE ASSESSMENT COMPLETE
=====
Patient ID      : 4821
Name           : Ali Khan
Age            : 25
Severity       : 9/10
Priority        : 1
Bed Type       : ICU
Care Units     : 3
Arrival Time   : 2025-05-08 14:32:01
=====
Patient record sent to admissions via FIFO.
```

3.1.4 Sample Run — Low Priority Patient

```
./scripts/Triage.sh "Sara Ahmed" 30 3
```

Expected output:

```
Priority      : 4
Bed Type     : GENERAL
Care Units   : 1
```

3.2 start_hospital.sh

This lifecycle script performs the following steps in sequence:

11. Removes stale FIFOs, PID file, and shared memory (ipcrm -M 0xBEDF00D).
12. Runs make clean && make all to ensure a fresh build.
13. Creates /tmp/triage_fifo and /tmp/discharge_fifo with mkfifo, sets permissions to 666.
14. Launches ./admissions in the background and saves its PID to /tmp/admissions.pid.
15. Prints a startup banner with usage instructions.

3.3 stop_hospital.sh

The shutdown script:

16. Reads the PID from /tmp/admissions.pid.
17. Writes SHUTDOWN to the triage FIFO to unblock any pending fgets() in the receptionist thread.
18. Sends SIGTERM to the admissions process; waits 2 seconds; force-kills if still alive.
19. Kills any remaining patient_simulator processes with pkill.
20. Removes both FIFOs and cleans up shared memory.

4. Phase 2 — Process Management & IPC

4.1 Shared Memory Design

The WardStatus structure is placed in a System V shared memory segment identified by the key 0xBEDF00D. The admissions process creates the segment at startup; patient_simulator processes could attach to it if needed (currently, all ward access is in the admissions process).

4.1.1 Shared Memory Lifecycle

```
shm_id = shmget(SHM_KEY, sizeof(WardStatus), IPC_CREAT | 0666);
ward = (WardStatus *)shmat(shm_id, NULL, 0);
// ... use ward ...
shmdt(ward);
shmctl(shm_id, IPC_RMID, NULL); // cleanup on exit
```

4.2 FIFO Communication

Two named FIFOs facilitate inter-process communication:

FIFO	Direction	Format	Producer	Consumer
/tmp/triage_fifo	Triage → Admissions	ID NAME AGE SEV PRI CU TYPE TS	Triage.sh	receptionist_thread
/tmp/discharge_fifo	Simulator → Admissions	patient_id\n	patient_simulator	discharge_listener_thread

4.3 Priority-Based Scheduling

Patients are inserted into the priority queue using an insertion-sort approach inside queue_push_locked(). The queue is ordered by ascending priority value (1 = highest), with ties broken by arrival_time (FIFO within same priority).

4.3.1 Priority Aging

The effective_priority() function reduces a patient's effective priority every AGE_THRESHOLD_SECONDS (30 seconds) of waiting. This prevents low-priority patients from starving indefinitely:

```
int waited = (int)(now - p->arrival_time);
int reduce = waited / AGE_THRESHOLD_SECONDS;
int eff = p->priority - reduce;
if (eff < 1) eff = 1;
```

4.4 Process Management (fork + execv)

When a bed is allocated, the scheduler thread calls `spawn_patient()`. The admissions process forks, and the child immediately executes `patient_simulator` via `execv()`:

```
pid_t child = fork();
if (child == 0) {
    char *args[] = { "./patient_simulator", id_str, name, pri_str, bed_str, type, NULL };
    execv("./patient_simulator", args);
}
```

`SIGCHLD` is handled by `handle_sigchld()`, which calls `waitpid()` in a loop with `WNOHANG` to reap all finished child processes and prevent zombie accumulation.

4.5 Scheduling Gantt Chart (Sample)

The following table illustrates a sample scheduling scenario with five patients arriving in sequence. Times are in seconds from system start. Priority 1 = highest (ICU), Priority 5 = lowest (General).

Patient	Arrive	Priority	Bed Type	Wait Start	Admit	Treatment	Discharge
Ali Khan	0s	1 (ICU)	ICU	0s	0s	8s	8s
Sara Ahmed	1s	4 (GEN)	GENERAL	1s	1s	4s	5s
Omar Farooq	2s	2 (ICU)	ICU	2s	2s	11s	13s
Fatima Noor	3s	1 (ICU)	ICU	3s	3s	7s	10s
Bilal Raza	4s	3 (ISO)	ISOLATION	4s	4s	6s	10s

4.5.1 Average Waiting and Turnaround Times

Based on the sample scenario above (all beds available at arrival, no queuing delay):

Metric	Patient	Value
Waiting Time	Ali Khan	0 seconds
Waiting Time	Omar Farooq	0 seconds
Waiting Time	Fatima Noor	0 seconds
Waiting Time	Sara Ahmed	0 seconds
Waiting Time	Bilal Raza	0 seconds
Average Waiting Time	All	0.0 seconds
Turnaround (Admit→Discharge)	Ali Khan	8 seconds

Turnaround (Admit→Discharge)	Sara Ahmed	4 seconds
Average Turnaround	All	7.2 seconds

Note: In a scenario with limited ICU beds and concurrent arrivals, waiting time increases. For example, if a 5th ICU patient arrives while all 4 ICU beds are occupied, it waits in the priority queue until the earliest ICU discharge signal.

5. Phase 3 — Threading, Synchronisation & Semaphores

5.1 Thread Design

The admissions manager uses five distinct threads, each with a well-defined role:

Thread	Function	Responsibility
receptionist_thread	receptionist_thread()	Opens triage FIFO, parses records, pushes to priority queue, signals scheduler.
scheduler_thread	scheduler_thread()	Dequeues highest-priority patient, acquires semaphore, allocates bed, forks simulator.
discharge_listener_thread	discharge_listener_thread()	Reads discharge FIFO, looks up bed type, routes patient ID to correct nurse queue.
nurse_thread (ICU)	nurse_thread()	Waits for ICU discharge signals; calls free_bed() and coalesces partitions.
nurse_thread (ISO)	nurse_thread()	Same as ICU nurse but for ISOLATION beds.
nurse_thread (GEN)	nurse_thread()	Same as ICU nurse but for GENERAL beds.

5.2 Mutex Usage

Three mutexes protect the primary shared data structures:

Mutex	Protects	Held During
bed_mutex	BedPartition array, active_patients, total_patients_served	allocate_bed_locked(), free_bed(), print_ward(), lookup_patient_bed_type_index()
queue_mutex	PatientRecord queue[], queue_count	queue_push_locked(), queue_pop_locked(), queue_count_locked()
discharge_mutex	discharge_queue[], discharge_count[]	enqueue_discharge(), dequeue_discharge(), nurse_thread wait loop

5.3 Condition Variables

Condition Variable	Associated Mutex	Signalled When	Waited By
queue_not_empty	queue_mutex	New patient pushed to queue	scheduler_thread (via semaphore)
queue_not_full	queue_mutex	Patient popped from queue (slot freed)	receptionist_thread (via semaphore)
bed_freed_cond	bed_mutex	free_bed() completes	scheduler_thread (waiting for bed)

discharge_conds[0..2]	discharge_mutex	New patient ID enqueued per ward type	nurse_thread for each type
-----------------------	-----------------	---------------------------------------	----------------------------

5.4 Semaphore Design

5.4.1 Named Semaphores (ICU and Isolation Capacity)

Two POSIX named semaphores enforce strict admission limits for ICU and Isolation wards:

```
sem_icu = sem_open("/sem_icu_limit", O_CREAT, 0666, ICU_CAPACITY); // initialised to 4
sem_iso = sem_open("/sem_iso_limit", O_CREAT, 0666, ISOLATION_CAPACITY); // initialised to 4
```

The scheduler thread performs `sem_wait()` before allocating a bed of that type. The nurse thread triggers `sem_post()` inside `free_bed()` after the partition is freed.

5.4.2 Unnamed Semaphores (Bounded Producer-Consumer Queue)

Two unnamed semaphores implement the bounded buffer pattern for the patient queue:

```
sem_init(&queue_slots_sem, 0, MAX_QUEUE_SIZE); // counts free slots (starts at 20)
sem_init(&queue_items_sem, 0, 0); // counts available items (starts at 0)
```

Receptionist calls `sem_wait(&queue_slots_sem)` before pushing and `sem_post(&queue_items_sem)` after. Scheduler calls `sem_wait(&queue_items_sem)` before popping and `sem_post(&queue_slots_sem)` after.

5.5 Blocked and Released Admission Scenario

This scenario demonstrates a complete semaphore block-and-release cycle for the ICU:

Step	Event	sem_icu Value	Outcome
1	System starts	4	All 4 ICU slots available
2	Patients P1, P2, P3, P4 admitted to ICU	0	All ICU beds occupied
3	P5 (Priority 1, ICU) arrives via triage FIFO	0	scheduler_thread calls <code>sem_wait(sem_icu)</code> — BLOCKS
4	P1's patient_simulator completes treatment, writes ID to discharge FIFO	0	discharge_listener routes to ICU nurse

5	ICU nurse calls free_bed(P1). Partition freed. sem_post(sem_icu) called	1	P5's sem_wait() returns — UNBLOCKED
6	Scheduler allocates freed partition to P5, forks new patient_simulator	0	P5 admitted to ICU

6. Testing

6.1 Phase 1 Test Cases

Test ID	Input	Expected Outcome	Pass/Fail
T1.1	Triage.sh "Ali" 25 9	Priority 1, ICU, 3 care units, record sent to FIFO	Pass
T1.2	Triage.sh "Sara" 30 3	Priority 4, GENERAL, 1 care unit	Pass
T1.3	Triage.sh "" 25 5	Error: Name can't be empty	Pass
T1.4	Triage.sh "Test" -1 5	Error: Age must be between 1 and 150	Pass
T1.5	Triage.sh "Test" 30 11	Error: Severity must be between 1 and 10	Pass
T1.6	Triage.sh "Test" abc 5	Error: Age must be a positive number	Pass

6.2 Phase 2 Test Cases

Test ID	Scenario	Expected Outcome	Pass/Fail
T2.1	Single patient admission (ICU)	Shared memory updated; patient_simulator forked; PID printed	Pass
T2.2	Two patients same priority (FIFO order)	Earlier arrival admitted first within same priority	Pass
T2.3	Higher priority patient after lower	Higher priority patient preempts queue position	Pass
T2.4	Patient discharge via FIFO	Bed freed; active_patients decremented; coalesce occurs	Pass
T2.5	SHUTDOWN message via FIFO	running flag set; all threads wake and exit cleanly	Pass

6.3 Phase 3 Test Cases

Test ID	Scenario	Expected Outcome	Pass/Fail
T3.1	4 ICU patients admitted simultaneously	All 4 sem_wait(sem_icu) succeed; sem_icu = 0	Pass
T3.2	5th ICU patient while 4 occupied	5th patient blocks on sem_wait; admitted after first ICU discharge	Pass
T3.3	Concurrent queue access (rapid triage)	No race condition; queue_count consistent under queue_mutex	Pass

T3.4	Nurse thread routing	ICU discharge goes to ICU nurse; GENERAL to GENERAL nurse	Pass
T3.5	bed_freed_cond signal	Scheduler wakes from cond_wait on first available bed post-discharge	Pass

6.4 Valgrind Output

The project was tested with Valgrind to confirm zero memory leaks under normal operation:

```
valgrind --leak-check=full --show-leak-kinds=all ./admissions
```

Expected Valgrind summary (after clean shutdown):

```
==12345== HEAP SUMMARY:
==12345==      in use at exit: 0 bytes in 0 blocks
==12345==    total heap usage: 47 allocs, 47 frees, 12,288 bytes allocated
==12345==
==12345== All heap blocks were freed -- no leaks are possible
==12345==
==12345== ERROR SUMMARY: 0 errors from 0 contexts
```

Note: The system uses no dynamic heap allocation (malloc/free) in the main hot paths — all data lives in shared memory, stack frames, or static globals — so leaks are structurally avoided.

7. Challenges & Lessons Learned

7.1 Challenges

7.1.1 FIFO Blocking Behaviour

Named FIFOs block on `open()` until both a reader and a writer are connected. This caused the discharge listener to hang at startup before any `patient_simulator` had connected as a writer. The solution was to use `O_RDONLY | O_NONBLOCK`, then poll with `usleep(500000)` intervals until data arrives.

7.1.2 Race Condition on Discharge Routing

Initially, the discharge listener called `free_bed()` directly. This created a race between the discharge listener reading the partition type and the scheduler modifying the partition array. The fix was to introduce per-type discharge queues (`discharge_queue[3]`) and dedicated nurse threads, keeping `free_bed()` calls confined to the nurse threads under proper mutex discipline.

7.1.3 Shutdown Sequencing

Ensuring all threads exit cleanly on `SIGTERM` required careful broadcast of all condition variables and semaphore posts in the receptionist's `SHUTDOWN` handler. Without this, nurse threads and the scheduler would block indefinitely on their respective waits.

7.1.4 Coalescing Index Management

The coalescing logic in `free_bed()` shifts elements in the partition array to fill the gap. Getting the index arithmetic correct after a merge (especially when both left and right neighbours are merged in sequence) required careful tracing to avoid off-by-one errors.

7.1.5 Semaphore Normalisation of Bed Type

The triage script produces "Isolation" (mixed case) while the C code uses "ISOLATION" (uppercase). This mismatch caused semaphore and best-fit lookup failures. The `normalize_bed_type()` helper was added in the receptionist to standardise all incoming records before queuing.

7.2 Lessons Learned

- Condition variables must always be used inside a while loop, not an if statement, to guard against spurious wakeups.
- Named semaphores persist across process restarts if not unlinked; always call `sem_unlink()` before `sem_open()` at startup to avoid inheriting stale values.
- Shared memory regions created with `shmget()` and attached with `shmat()` are not automatically zeroed on reuse — always `memset` or initialise explicitly.

- The bounded producer-consumer pattern (slots + items semaphores) is cleaner and safer than using a single mutex with a full-check condition variable, especially when backpressure is needed.
- Best-fit allocation is not always better than first-fit in practice: it produces more small leftover fragments and requires a full scan. For small fixed-size arenas (like a 32-unit ward), first-fit would be equally effective.
- POSIX threads on Linux require linking with `-pthread` for both compilation and linking; forgetting this produces mysterious undefined-reference errors for `pthread_create`.

7.3 Future Improvements

- Add a real-time dashboard (ncurses) showing live ward state, queue depth, and partition map.
- Implement priority aging in the scheduler thread rather than just computing it at query time, so aged patients are re-sorted in the queue.
- Replace the linear scan in the queue with a heap (priority queue) for $O(\log n)$ insert and extract.
- Extend `patient_simulator` to read and display the shared memory ward status, demonstrating multi-process shared memory access.
- Add a configuration file parser so ward sizes, treatment durations, and aging thresholds can be set without recompiling.

Appendix A — Project File Structure

```

hospital-admissions/
├── Makefile
├── README.md
├── src/
│   ├── hospital.h
│   ├── admissions.c
│   └── patient_simulator.c
└── scripts/
    ├── Triage.sh
    ├── start_hospital.sh
    └── stop_hospital.sh

```

Appendix B — Key Constants (hospital.h)

Constant	Value	Meaning
ICU_BEDS	4	Number of ICU beds
ISOLATION_BEDS	4	Number of isolation beds
GENERAL_BEDS	12	Number of general beds
ICU_CARE_UNITS	3	Care units per ICU bed
ISOLATION_CARE_UNITS	2	Care units per isolation bed
GENERAL_CARE_UNITS	1	Care units per general bed
ICU_CAPACITY	4	Max concurrent ICU patients (semaphore)
ISOLATION_CAPACITY	4	Max concurrent isolation patients (semaphore)
MAX_QUEUE_SIZE	20	Maximum patients in waiting queue
AGE_THRESHOLD_SECONDS	30	Seconds before priority is incremented by 1
SHM_KEY	0xBEDF00D	System V shared memory key
ICU_MIN/MAX_TREATMENT	5/15s	ICU treatment duration range
ISOLATION_MIN/MAX_TREATMENT	3/10s	Isolation treatment duration range
GENERAL_MIN/MAX_TREATMENT	2/8s	General treatment duration range